

Session 17: Pandas DataFrame

(Complete Classroom Notes)

1. Introduction to DataFrames

A **DataFrame** is the most important object in Pandas. While a **Series** stores 1D data (like a single column), a **DataFrame** is used to store **Tabular Data** (rows and columns).

Concept Breakdown:

- **Table = DataFrame:** Any data structure consisting of rows and columns is treated as a DataFrame in Pandas.
- **Single Column = Series:** Any single column within a DataFrame is a Series.
- **Single Row = Series:** Interestingly, every single row in a DataFrame is also treated as a Series.
- **Structure:** It is a 2D labeled data structure with columns of potentially different types.

2. Creating DataFrames

The teacher explained three primary ways to create a DataFrame.

A. Using a 2D List

You can create a DataFrame by passing a list within a list (2D list).

Code Snippet:

```
import pandas as pd

# Data: [IQ, Marks, Package]
student_data = [
    [100, 80, 10],
    [90, 70, 7],
    [120, 100, 14],
    [80, 50, 2]
]

df = pd.DataFrame(student_data, columns=['IQ', 'Marks', 'Package'])
print(df)
```

Line-by-line Explanation:

1. `import pandas as pd`: Imports the pandas library.

2. `student_data = [...]`: A 2D list where each inner list represents one student's record (a row).
3. `pd.DataFrame(student_data, columns=[...])`: The main function to create the table. We pass the data and explicitly name the columns.
4. `print(df)`: Displays the table with automatically generated index (0, 1, 2, 3).

B. Using a Dictionary

This is often easier because keys automatically become column names.

Code Snippet:

```
students = {  
    'IQ': [100, 90, 120, 80],  
    'Marks': [80, 70, 100, 50],  
    'Package': [10, 7, 14, 2]  
}
```

```
df_dict = pd.DataFrame(students)
```

Line-by-line Explanation:

1. `students = { 'IQ': [...], ... }`: A dictionary where keys are column headers and values are lists of data for that column.
2. `pd.DataFrame(students)`: Converts the dictionary into a table. No need to pass a separate columns list.

C. Importing from External Files (Real-world Method)

In professional projects, we usually import data using `read_csv`.

Code Snippet:

```
# Importing Movie Data  
movies = pd.read_csv('movies.csv')
```

```
# Importing IPL Matches Data  
ipl = pd.read_csv('ipl-matches.csv')
```

3. Important Attributes of DataFrames

Attributes provide metadata about the DataFrame without changing it.

1. **.shape**: Returns a tuple (rows, columns).
 - Example: `movies.shape` might return (1629, 18), meaning 1629 movies and 18 info columns.

2. **.dtypes**: Shows the data type of every column.
 - *Note*: In Pandas, "Object" usually means String.
3. **.index**: Returns the index range (e.g., RangeIndex 0 to 1629).
4. **.columns**: Returns an object containing all column names.
5. **.values**: Converts the DataFrame into a 2D **NumPy Array**.

4. Useful Methods (Functions)

A. Previewing Data

- **head()** / **tail()**: View top or bottom 5 rows. Use head(2) for top 2.
- **sample(n)**: Picks n random rows.
 - *Tip*: Use this to check for "bias" in data (e.g., if data is sorted by marks, head only shows toppers).

B. High-Level Summary

- **info()**: The teacher's favorite. It shows:
 - Total entries and column names.
 - Count of "Non-Null" values (helps find missing data).
 - Data types and Memory Usage (RAM occupied).
- **describe()**: Provides a mathematical summary (Mean, Std Dev, Min, Max, Percentiles) for **Numerical Columns** only.

C. Data Cleaning Checks

- **isnull().sum()**: Tells you exactly how many missing values are in each column.
- **uplicated().sum()**: Tells you how many rows are exact repeats (useless for analysis).
- **rename()**: Used to change column names.
df.rename(columns={'Marks': 'Percent', 'Package': 'LPA'}, inplace=True)
 - **Reasoning**: inplace=True makes the change permanent in the original variable.

5. Mathematical Functions

You can run math on the whole DataFrame or specific axes.

Code Snippet:

```
# Column-wise sum (Default)
df.sum()
```

```
# Row-wise sum
df.sum(axis=1)
```

Line-by-line Explanation:

1. `df.sum()`: Adds all values in each column (IQ sum, Marks sum, etc.). Default is `axis=0`.
2. `df.sum(axis=1)`: Adds values across a row (e.g., IQ + Marks + Package for Student 1).
3. *Note*: Functions like `mean()`, `median()`, `std()`, and `var()` work the same way.

6. Selecting Data (Slicing/Indexing)

A. Selecting Columns

- **Single Column**: `df['col_name']` (Returns a **Series**).
- **Multiple Columns**: `df[['col1', 'col2']]` (Returns a **DataFrame**).
 - *Note*: We use double brackets `[[ ]]` because we are passing a list of column names.

B. Selecting Rows (`iloc` vs `loc`)

- **`iloc` (Index Location)**: Uses integer positions.
 - `df.iloc[0]`: 1st row.
 - `df.iloc[0:5]`: 1st five rows.
 - `df.iloc[0:3, 0:2]`: First 3 rows AND first 2 columns.
- **`loc` (Label Location)**: Uses row labels (like student names if set as index).
 - `df.loc['Nitish']`: Fetches row where index label is 'Nitish'.

7. Filtering Data (Boolean Indexing)

This is used to find rows based on logic.

Example 1: Find all IPL Final winners

```
winners = ipl[ipl['match_number'] == 'Final'][['season', 'winning_team']]
```

- **Explanation**: We filter rows where `match_number` is 'Final', then only show `season` and `winning_team` columns.

Example 2: Multiple Conditions (CSK wins in Kolkata)

```
mask1 = ipl['city'] == 'Kolkata'
mask2 = ipl['winning_team'] == 'Chennai Super Kings'
result = ipl[mask1 & mask2]
```

- **Reasoning**: We use the `&` (Bitwise AND) operator because we are comparing series of True/False values.

8. Adding/Modifying Columns

You can create new columns based on existing ones.

Example: Extracting Lead Actor

```
def get_lead(text):  
    return text.split('|')[0]  
  
movies['lead_actor'] = movies['actors'].apply(get_lead)
```

Line-by-line Explanation:

1. `def get_lead(text)`: A function that splits a string by the '|' symbol and takes the first name.
2. `movies['actors'].apply(get_lead)`: Applies this logic to every row in the 'actors' column.
3. `movies['lead_actor'] = ...`: Saves the results into a brand new column.

9. Memory Optimization (The astype method)

The teacher emphasized reducing RAM usage for large datasets.

Code Snippet:

```
# Convert 64-bit int to 32-bit to save space  
ipl['ID'] = ipl['ID'].astype('int32')  
  
# Convert repetitive strings to 'category' type  
ipl['season'] = ipl['season'].astype('category')  
ipl['team1'] = ipl['team1'].astype('category')
```

Key Points:

- **Why?:** int64 takes more space than int32. If your numbers are small, int32 saves RAM.
- **Category Type:** If a column has many repeating strings (like Team Names), converting to category significantly reduces the memory footprint (sometimes up to 80-90%).

Important Tips & Highlights:

- **Hierarchy:** DataFrame > Series > Scalar.
- **Axis Rule:** axis=0 is Vertical (Columns), axis=1 is Horizontal (Rows).
- **Missing Values:** Always use `info()` or `isnull().sum()` first thing after loading data to see how "dirty" it is.
- **Object Type:** If a column is shown as object, it usually contains strings or mixed data.

End of Notes.